Step 1: The server-side checks

Check	Purpose
Authentication	Confirm the request comes from an authenticated user.
Nonce verification	Protect cookie-authenticated requests against CSRF attacks. A nonce is not a substitute for authorization.
Capability check	Confirm the user is allowed to edit the specific article and perform the requested action.
Input validation	Confirm the article ID and requested status are valid.
State and transition rules	Prevent unsupported status changes and enforce editorial workflow rules.
Error handling	Reject invalid requests without exposing internal details.

Step 2: Example WordPress REST API endpoint

This example registers a REST endpoint for changing a post's status. It uses WordPress's REST authentication and permission mechanisms, then validates the article and requested status before updating anything

```
<?php

add_action('rest_api_init', function () {
   register_rest_route('bcm/v1', '/article-status', [
      'methods'            => 'POST',
      'callback'           => 'bcm_update_article_status',
      'permission_callback' => 'bcm_can_update_article_status',
      'args'               => [
         'id' => [
            'required'         => true,
            'validate_callback' => function ($value) {
               return filter_var(
```

```
                $value,
                FILTER_VALIDATE_INT
            ) !== false && (int) $value > 0;
        },
      ],
      'status' => [
        'required' => true,
        'type'     => 'string',
        'enum'     => [
          'draft',
          'pending',
          'publish',
        ],
      ],
    ],
  ]);
});

/**
 * Check authorization before performing the update.
 */
function bcm_can_update_article_status(WP_REST_Request $request) {
  $post_id = absint($request->get_param('id'));

  // REST authentication must be configured correctly.
  if (!is_user_logged_in()) {
    return new WP_Error(
      'rest_forbidden',
      'Authentication is required.',
      ['status' => 401]
    );
  }
```

```
    // Require the appropriate capability for the article.
    if (
        $post_id < 1 ||
        !current_user_can('edit_post', $post_id)
    ) {
        return new WP_Error(
            'rest_forbidden',
            'You are not allowed to edit this article.',
            ['status' => 403]
        );
    }

    // Require permission to publish when requested.
    if (
        $request->get_param('status') === 'publish' &&
        !current_user_can('publish_posts')
    ) {
        return new WP_Error(
            'rest_forbidden',
            'You are not allowed to publish posts.',
            ['status' => 403]
        );
    }

    return true;
}

/**
 * Validate the target and update its status.
 */
function bcm_update_article_status(WP_REST_Request $request) {
```

```
$post_id = absint($request->get_param('id'));
$status  = $request->get_param('status');

$post = get_post($post_id);

if (!$post) {
    return new WP_Error(
        'rest_post_not_found',
        'Article not found.',
        ['status' => 404]
    );
}

if ($post->post_type !== 'post') {
    return new WP_Error(
        'rest_invalid_post_type',
        'Only standard posts are supported.',
        ['status' => 400]
    );
}

$allowed_statuses = ['draft', 'pending', 'publish'];

if (!in_array($status, $allowed_statuses, true)) {
    return new WP_Error(
        'rest_invalid_status',
        'Invalid article status.',
        ['status' => 400]
    );
}

// Enforce the capability required for publishing.
```

```
    if (
        $status === 'publish' &&
        !current_user_can('publish_post', $post_id)
    ) {
        return new WP_Error(
            'rest_forbidden',
            'You cannot publish this article.',
            ['status' => 403]
        );
    }

    $result = wp_update_post([
        'ID'          => $post_id,
        'post_status' => $status,
    ], true);

    if (is_wp_error($result)) {
        return new WP_Error(
            'rest_update_failed',
            'The article status could not be updated.',
            ['status' => 500]
        );
    }

    return rest_ensure_response([
        'success' => true,
        'id'      => $result,
        'status'  => get_post_status($result),
    ]);
}
```

Step 3: The security checks

- ✓ register_rest_route() - creates a server-side endpoint.
- ✓ permission_callback - checks whether the caller is authorized. The browser's UI is not used as evidence of permission.
- ✓ current_user_can('edit_post', $post_id) - checks the user's capability against the particular post, rather than relying on a role name alone.
- ✓ The allowed status list prevents arbitrary status strings from being submitted.
- ✓ The post-type check prevents the endpoint from changing unrelated content types.
- ✓ wp_update_post() performs the update using WordPress's own API and error handling.

Two tests that must be rejected

Test A: Unauthenticated request

i. Send a direct POST request without valid authentication, even if the caller knows the article ID.

ii. Expected: the request is rejected, typically with HTTP 401 or the applicable WordPress REST authentication error. The article status remains unchanged

Test B: Authenticated but unauthorized request

i. Sign in as a user who lacks permission to edit the target article, then submit a valid article ID and publish status.

ii. Expected: HTTP 403 or the applicable authorization error. The article must not be published or otherwise modified.

Step 5: How I would raise the issue with the developer

I would explain that client-side visibility is a usability feature, not an authorization boundary. I would demonstrate that a direct request can bypass the hidden button and show the resulting risk without modifying production content.

Before release, I would require server-side authentication and authorization, correct nonce handling for cookie authentication, input validation, appropriate status-transition rules, and automated tests proving that unauthorized requests leave the article unchanged. I would also ask for a code review and verification in a staging environment.